AGENZIA VIAGGI MARE SOLE

TRACCIA:

L’agenzia Viaggi Mare-Sole gestisce prenotazioni di barche provviste di cabine e condotte da skipper. Nella prenotazione i clienti devono indicare il proprio nominativo, il numero di cabine e la data di partenza costituita da giorno, mese e anno. Progettare una soluzione al problema che per ogni prenotazione trattata fornisca in output un messaggio di prenotazione effettuata o di non disponibilità della barca richiesta. Si vuole inoltre che, inserito in input il nominativo di uno skipper, si forniscano in output i dati della barca che conduce.

Per la soluzione del problema, sviluppare le seguenti fasi:
a) Analizzare il problema chiarendo le sue specifiche e formulando eventuali ipotesi necessarie per la soluzione. Individuare i dati di ingresso (dominio di definizione) e fornire un campione. Individuare i dati di uscita o risultati (dominio di definizione) e fornire il campione associato al campione di ingresso. Sintetizzare l'analisi indicando l'incognita, i dati di ingresso e la condizione. b) Progettare la strategia di soluzione individuando una scomposizione del problema in sottoproblemi e rappresentarla mediante uno schema (ad es. l’albero di sviluppo).
c) Descrivere la strategia di soluzione mediante un linguaggio di descrizione.
d) Eseguire il trace con uno o più campioni di dati.

PUNTO A

CHIARIFICA:

Creare un programma che gestisca l'approvvigionamento di barche, le quali contengono delle cabine dove soggiornano gli acquirenti, condotte da uno skipper, gestite in un array di record.
La prenotazione del cliente sarà gestita in una funzione dove l'utente inserirà in input il suo nome, il numero di cabine che vuole acquistare e la data di partenza, specifica di mese, anno e giorno, quest'ultima gestita in un altro record. Inoltre in una nuova funzione il programma dovrà presentare in output i dati della barca gestiti da uno skipper definito in input dall'utente.
In output il programma dovrà mostrare, con un controllo sulla data, se la prenotazione è stata effettuata con successo o meno.
Si ipotizza che l'utente inserisca la data nel range corretto autonomamente senza dover controllando solo se la data fuoriesca dal range dei regolari giorni del mese e non se esista per quel mese, si effettua il controllo solo per chiarire se quella data è libera o è stata prenotata da un altro cliente attraverso un flag. Inoltre gli skipper e le barche condotte da essi hanno come dato il numero di cabine e il nome della barca, inoltre verrà gestito in output un messaggio di errore in caso lo skipper inserito in ricerca non esista. Si ipotizza che

uno skipper è assegnato ad una singola barca e non esistono barche con lo stesso nome, per questo non si effettueranno controlli su questa circostanza.
Si ipotizza che le cabine abbiano un numero massimo delimitato da una costante e saranno controllate solo per verificare se c'è disponibilità per l'utente che effettua l'ordine senza defalcare la loro cifra dopo l'ordine effettuato.

DATI:

DATI DI INPUT:

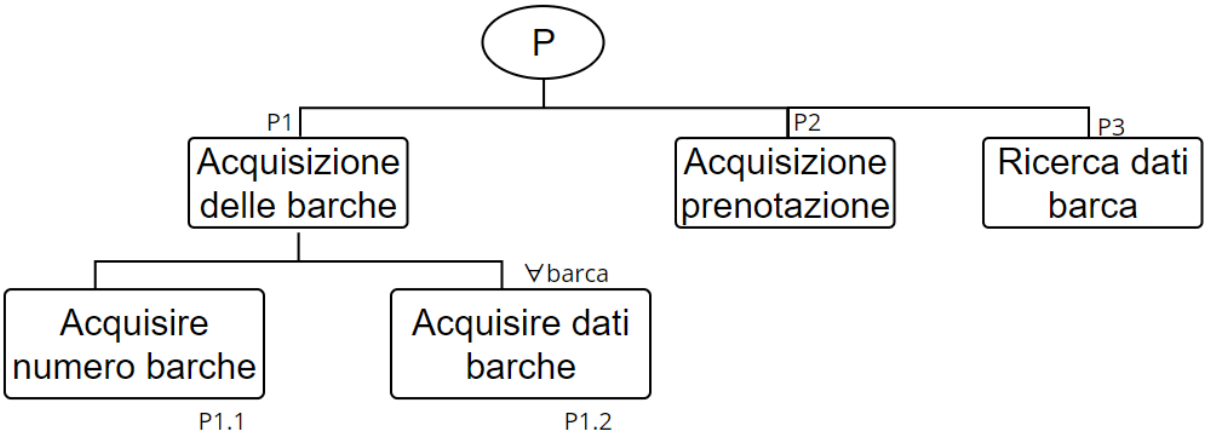
DATI	VARIABILI
Array Clienti	cliente (array di record)
-Nome Cliente	cliente[i].nome (stringa char)
-Numero Cabine da Acquistare	cliente[i].ncabine (intero > 0 e < CABINEMAX)
-Data	cliente[i].data (record)
--Giorno	data.g (intero > 0 e < 32)
--Mese	data.mese (intero > 0 e < 13)
--Anno	data.anno (intero >= ANNO ATTUALE)
Numero Barche	numbarche (intero)
Array Barca	barca (array di record)
-Skipper	barca[i].skipper (stringa char)
-Nome Barca	barca[i].nbarca (stringa char)
-Numero Cabine	barca[i].numcab (intero > 0 e < CABINEMAX)
Nome Skipper da Ricercare	ric_skipper (stringa char)

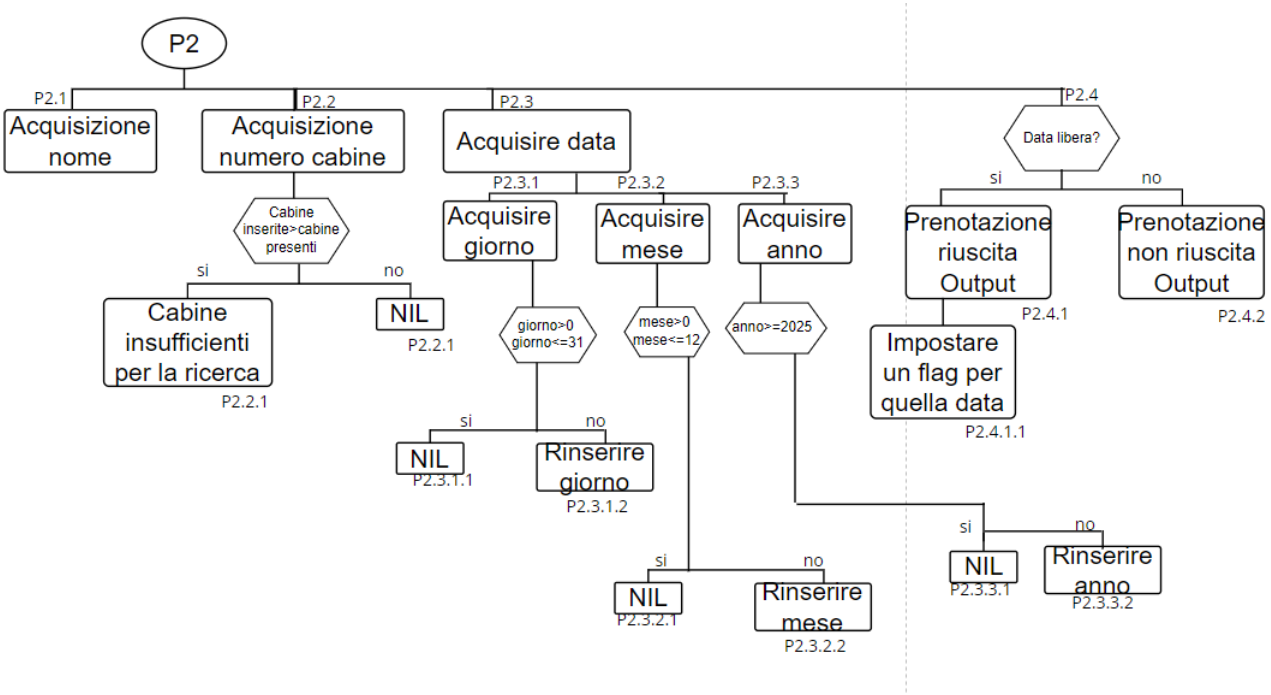
DATI DI OUTPUT:

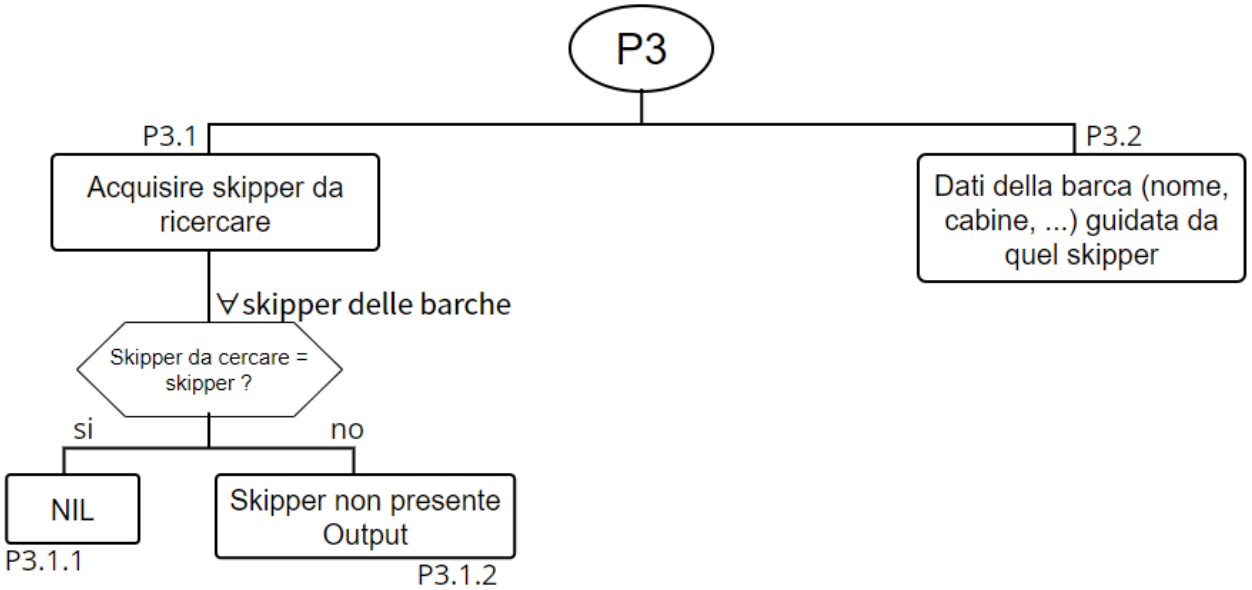
DATI	VARIABILI
Esito della prenotazione effettuata con successo	Prenotazione riuscita
Esito della prenotazione non riuscita	Prenotazione non disponibile
Dati barche filtrati per skipper	barca[i].nbarca, barca[i].numcab, barca[i].skipper
Esito cabine non disponibili	Cabine non disponibili, bye bye
Messaggio di errore nella ricerca di barche	Barca non disponibile per questo conducente o conducente inesistente

PUNTO B

SCOMPOSIZIONE IN SOTTOPROBLEMI/ ALBERO DI DECOMPOSIZIONE:

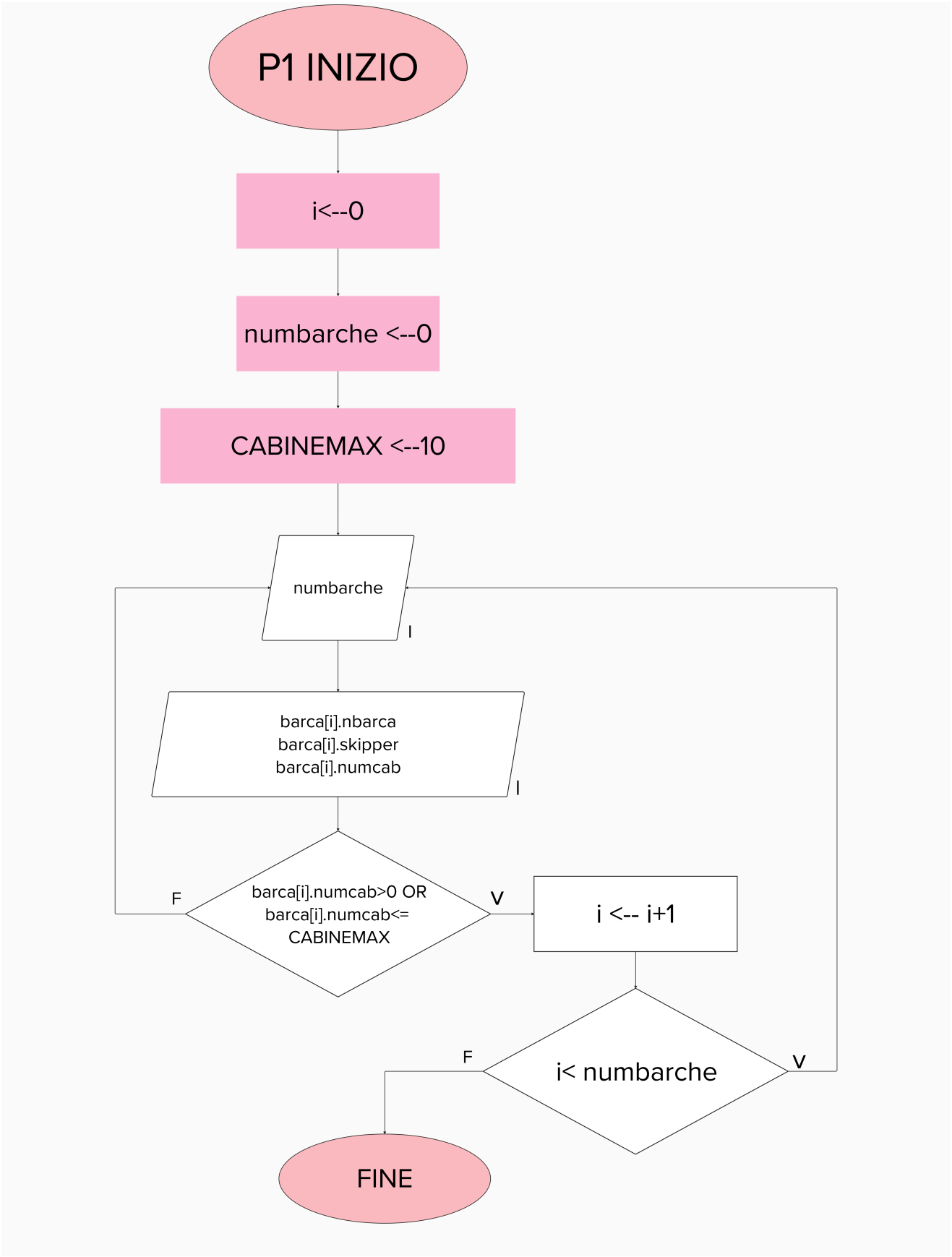


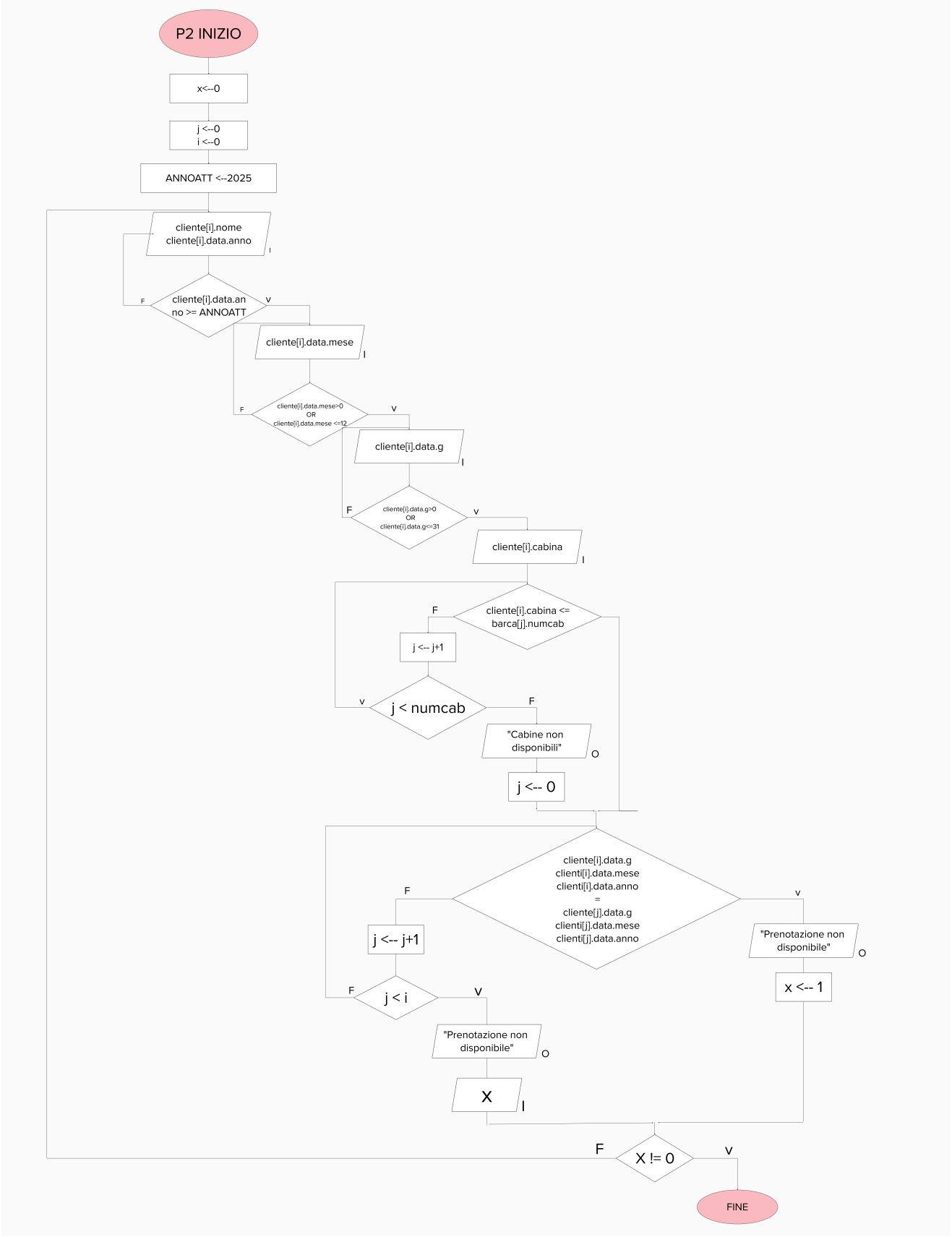


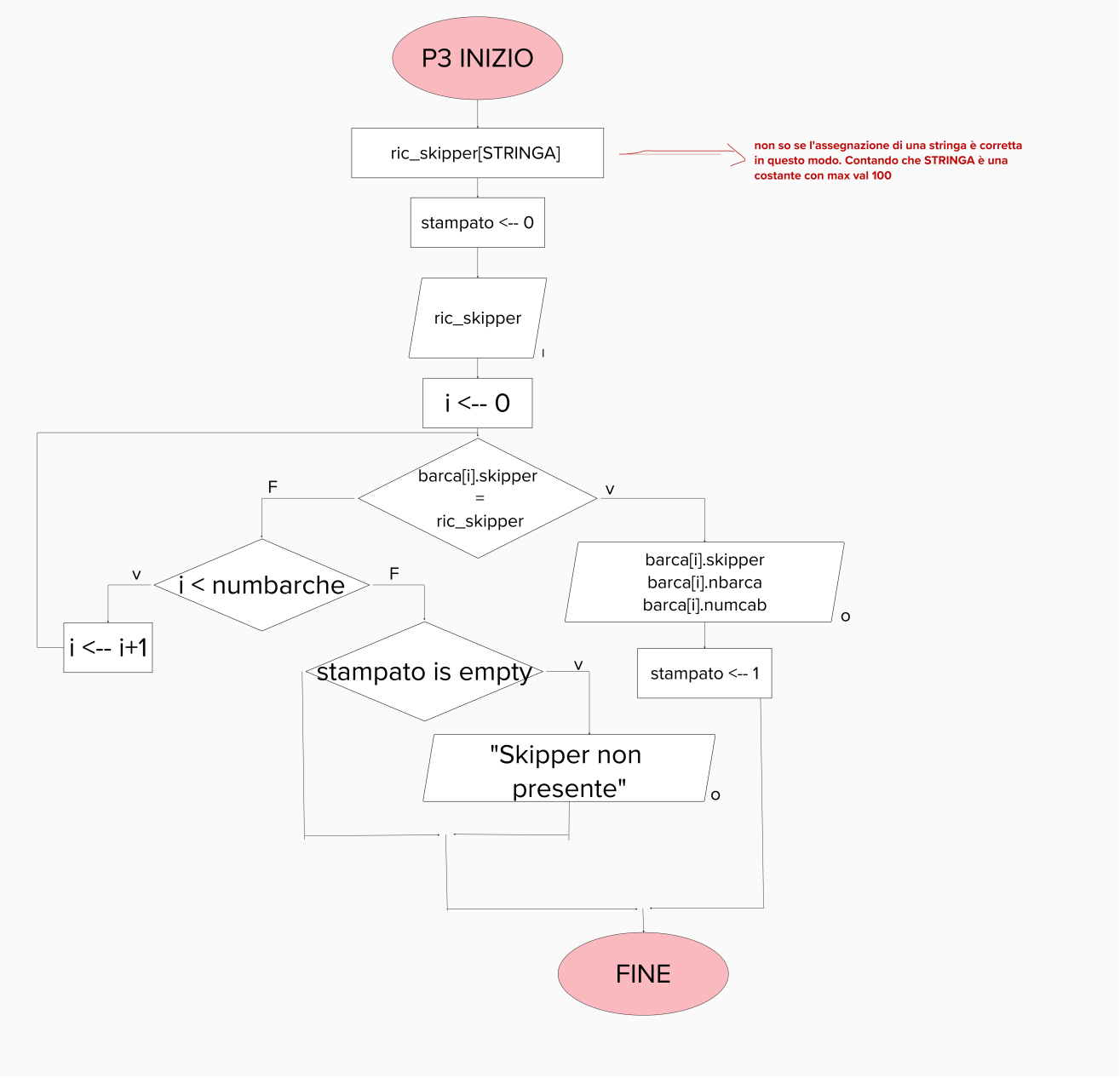


PUNTO C

FLOWCHART:


P1 INIZIO
i<--0
numbarche <--0
CABINEMAX <--10
numbarche
I
barca[i].nbarca
barca[i].skipper
barca[i].numcab
I
F
barca[i].numcab>0 OR
barca[i].numcab<=
CABINEMAX
V
i <-- i+1
F
i< numbarche
V
FINE


P2 INIZIO
x<--0
j <--0
i <--0
ANNOATT <--2025
cliente[i].nome
cliente[i].data.anno
I
cliente[i].data.an
no >= ANNOATT
F
V
cliente[i].data.mese
I
cliente[i].data.mese>0
OR
cliente[i].data.mese <=12
F
V
cliente[i].data.g
I
cliente[i].data.g>0
OR
cliente[i].data.g<=31
F
V
cliente[i].cabina
I
cliente[i].cabina <=
barca[j].numcab
F
j <-- j+1
j < numcab
V
F
"Cabine non
disponibili"
O
j <-- 0
cliente[i].data.g
clienti[i].data.mese
clienti[i].data.anno
=
cliente[j].data.g
clienti[j].data.mese
clienti[j].data.anno
F
V
j <-- j+1
"Prenotazione non
disponibile"
O
x <-- 1
j < i
F
V
"Prenotazione non
disponibile"
O
X
I
X != 0
F
V
FINE



PUNTO D

CODICE:

```
/*****************************************************************************
***

Agenzia viaggi

*****************************************************************************
***/
#include <stdio.h>
#include <string.h>

#define ANNOATT 2025
#define CABINEMAX 10
```

```
#define STRINGA 80

//strutture
typedef struct{
    int g;
    int mese;
    int anno;
}data_t;

typedef struct{
    char nome[STRINGA];
    int ncabine;
    data_t d;
}cliente_t;

typedef struct{
    char skipper[STRINGA];
    int numcab;
    char nbarca[STRINGA];
}barca_t;

//dichiarazione funzioni
int caricamento_barca(barca_t barca[]);
void visualizzabarca(barca_t barca[], int numbarche);
void prenotazione(cliente_t cliente[], int numbarche, barca_t barca[]);
void datibarca(barca_t barca[], int numbarche, char* ric_skipper);

int main()
{
    barca_t b[10];
    int numero_barche=0;
    char rskipper[STRINGA];

    numero_barche=caricamento_barca(b);
    cliente_t c[10];
    prenotazione(c,numero_barche,b);

    printf("\nInserire skipper da cercare per vedere i dati della barca: ");
    scanf("%s", rskipper);
    datibarca(b,numero_barche,rskipper);
    return 0;
}
```

```
//funzione P1
int caricamento_barca(barca_t barca[]){
    int numbarche=0;
    printf("Inserire il numero delle barche presenti al porto: ");
    scanf("%d", &numbarche);
    for(short i=0;i<numbarche;i++){
        printf("\n Inserire il nome della barca numero %hd: ", i+1);
        scanf("%s", barca[i].nbarca);
        printf("\n Inserire lo skipper della barca numero %hd: ", i+1);
        scanf("%s", barca[i].skipper);
        do{
            printf("\n Inserire il numero di cabine presenti nella barca
numero %hd: ", i+1);
            scanf("%d", &barca[i].numcab);
        }while(barca[i].numcab<0 || barca[i].numcab > CABINEMAX);
    }
}

//prova di visualizzazione barche
/*
void visualizzabarca(barca_t barca[], int numbarche){
    for(short i=0;i<numbarche;i++){
        printf("\n I dati della barca numero %hd sono: ", i+1);
        printf("\n %s %s %d", barca[i].nbarca, barca[i].skipper,
barca[i].numcab);
    }
}*/

//funzione P2
void prenotazione(cliente_t cliente[], int numbarche, barca_t barca[]){
    short i=0, j=0, flag=0,x=0;
    do{
        printf("\nPer eseguire la prenotazione della sua barca procedere ad
inserire il vostro nome: ");
        scanf("%s", cliente[i].nome);
        printf("\nPer continuare con la prenotazione procedere ad inserire
la data di partenza, \nASSICURAZRSI DI INSERIRLA CORRETTAMENTE");
        do{
            printf("\nInserire il giorno: ");
            scanf("%d", &cliente[i].d.g);
        }while(cliente[i].d.g<1 || cliente[i].d.g>31); // in maniera tale da
non accettare lo 0
```

```
        do{
            printf("\nInserire il mese: ");
            scanf("%d", &cliente[i].d.mese);
        }while(cliente[i].d.mese<1 || cliente[i].d.mese>12);

        do{
            printf("\nInserire l'anno: ");
            scanf("%d", &cliente[i].d.anno);
        }while(cliente[i].d.anno<ANNOATT);

        printf("\nFornire numero cabine da prenotare: ");
        scanf("%d", &cliente[i].ncabine);
        for(j=0;j<numbarche;j++){
            if(cliente[i].ncabine>barca[j].numcab) flag=1; // così potremmo
uscire direttamente dal programma
            else flag=0;
        }
        if(flag==1){
            printf("\nCabine non disponibili, bye bye");
            x=1; //In modo tale che esca dal ciclo
        }

        //controllo sulla data per vedere se coincide con le altre presenti
nel codice
        j=0;
        for(j;j<i;j++){
            if((cliente[i].d.g==cliente[j].d.g)||
(cliente[i].d.mese==cliente[j].d.mese)||
(cliente[i].d.anno==cliente[j].d.anno)){
                printf("\nPrenotazione non disponibile in questa data");
                x=1;
            }
        }
        if(x!=1) printf("\nPrenotazione riuscita con successo");

        if(x==0){
            printf("\nSe si vuole continuare a prenotare un altro cliente
premere '0' altrimenti qualunque valore ");
            scanf("%hd", &x);
            if(x==0) i++;
        }
    }while(x==0);
}
```

```
//Funzione P3
void datibarca(barca_t barca[], int numbarche, char* ric_skipper){
    short stampato=0;
    for(short i=0;i<numbarche;i++){
        if(strncmp(barca[i].skipper, ric_skipper, STRINGA)==0){
            printf("\n I dati della barca guidati dallo skipper %s sono: ",
ric_skipper);
            printf("\n %s %s %d", barca[i].nbarca, barca[i].skipper,
barca[i].numcab);
            stampato=1;
        }
    }
    if(!stampato) printf("Skipper non presente");
}
```